UNIVERSIDADE DO ESTADO DE SANTA CATARINA - UDESC
Centro de Ciências Tecnológicas - CCT
Bacharelado em Ciência da Computação

RELATÓRIO DE EXPERIMENTO

Análise de Complexidade Algorítmica em Estruturas de Dados Balanceadas

Árvore AVL, Rubro-Negra e Árvore B

Disciplina	Estruturas de Dados e Algoritmos II (EDA II)
Tema	Análise de Estruturas de Dados Balanceadas
Estruturas Avaliadas	Árvore AVL, Rubro-Negra, Árvore B (ordens 1, 5 e 10)
Operações	Inserção e Remoção com Balanceamento
Métrica de Esforço	Comparações + Modificações Estruturais (rotações/splits)
Tamanho dos Dados	N = 100 a 10.000 (incremento de 100)
Amostras por N	10 conjuntos (média estatística)

Gabriel Petrentchuk
Julio Cesar Crestani Peretti
Rafael Horst Ruthes

Professor Alan Rodrigo Leite

Joinville, 29 de Junho de 2026

1. Introdução

Este relatório apresenta os resultados de um experimento computacional que analisa o desempenho de três estruturas de dados balanceadas amplamente utilizadas em ciência da computação: a Árvore AVL, a Árvore Rubro-Negra e a Árvore B. O foco da análise recai sobre as operações de inserção e remoção de nós, considerando o custo adicional introduzido pelo rebalanceamento estrutural.

Estruturas de dados balanceadas são fundamentais para garantir desempenho logarítmico em operações de busca, inserção e remoção. Enquanto uma árvore binária de busca simples pode degenerar para O(n) no pior caso, as estruturas balanceadas mantêm a altura da árvore controlada através de rotações, recolorações e divisões de nós, garantindo complexidade O(log n) em todos os casos.

O experimento foi implementado em linguagem C, utilizando chaves aleatórias únicas para simular o caso médio. Para cada tamanho de entrada N (variando de 100 a 10.000, em incrementos de 100), foram realizadas 10 amostragens independentes, e a média dos esforços foi registrada em um arquivo CSV. Os gráficos resultantes permitem visualizar e comparar o comportamento assintótico de cada estrutura.

2. Metodologia

2.1 Implementação

O experimento foi implementado em linguagem C, organizado nos seguintes módulos:

- avl.c / avl.h: implementação da Árvore AVL com rastreamento de comparações e rotações;
- rubro_negra.c / rubro_negra.h: implementação da Árvore Rubro-Negra com rastreamento de comparações e rotações/recolorações;
- arvore_b.c / arvore_b.h: implementação parametrizável da Árvore B, instanciada com ordens 1, 5 e 10, rastreando comparações e splits/merges;
- main.c: orquestração do experimento, geração das chaves, execução das amostras e exportação dos resultados em CSV.

2.2 Protocolo Experimental

O protocolo seguiu os seguintes passos para cada valor de N:

- Geração de um vetor de N chaves inteiras distintas (1 a N);
- Embaralhamento usando o algoritmo de Fisher-Yates (para garantir ordem aleatória uniforme);
- Inserção sequencial das N chaves em cada estrutura, com contagem acumulada de esforço;
- Remoção de todas as chaves em ordem diferente (segundo vetor embaralhado independentemente), com contagem acumulada de esforço;
- Repetição de 10 amostras independentes por valor de N;
- Cálculo da média aritmética das 10 amostras e registro no CSV.

2.3 Métrica de Esforço

A métrica de esforço computacional adotada é definida como a soma de duas componentes:

Esforço = Comparações + Modificações Estruturais

As Comparações contabilizam cada operação de comparação entre chaves realizada durante a travessia da árvore. As Modificações Estruturais representam as rotações (AVL e Rubro-Negra) e os splits/merges (Árvore B) realizados para rebalancear a estrutura após cada inserção ou remoção. Esta métrica captura tanto o custo de localização quanto o custo de manutenção do balanceamento.

2.4 Complexidade Teórica das Estruturas

Estrutura	Inserção	Remoção	Balanceamento
Árvore AVL	O(log n)	O(log n)	Rotações simples/duplas (até 2 por op.)
Rubro-Negra	O(log n)	O(log n)	Rotações + recoloração (amortizado O(1))
Árvore B (t=1)	O(log n)	O(log n)	Splits/merges frequentes (nós pequenos)
Árvore B (t=5)	O(log n)	O(log n)	Splits/merges moderados
Árvore B (t=10)	O(log n)	O(log n)	Splits/merges menos frequentes

Tabela 1 – Complexidade teórica das operações e caracterização do balanceamento de cada estrutura.

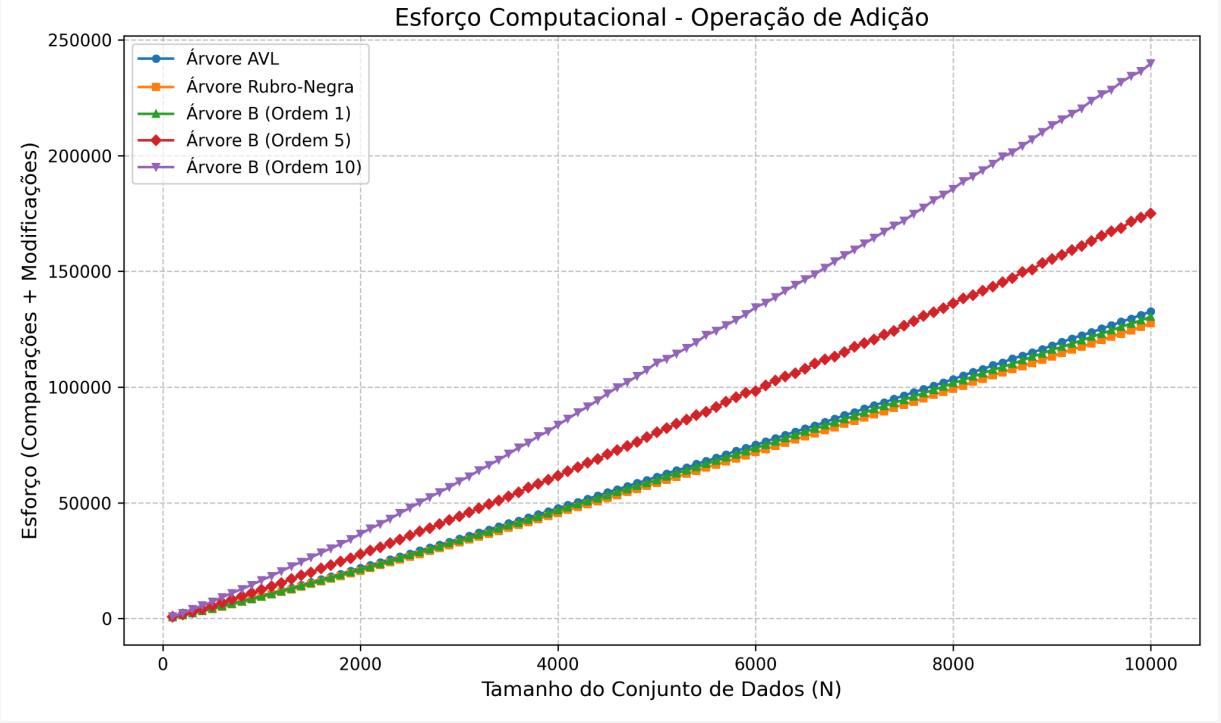
2.5 Ambiente de Execução

O programa foi compilado com o compilador GCC utilizando a flag de otimização -O3, assegurando que o custo de execução reflita o comportamento algorítmico das estruturas e não artefatos de tradução do compilador. A geração dos gráficos foi realizada em Python 3 com as bibliotecas pandas e matplotlib.

3. Resultados

Os gráficos a seguir apresentam o esforço computacional médio (Comparações + Modificações Estruturais) das operações de inserção e remoção para cada estrutura de dados, em função do tamanho N do conjunto de dados.

3.1 Operação de Inserção



[Gráfico 1: Esforço Computacional – Operação de Inserção]

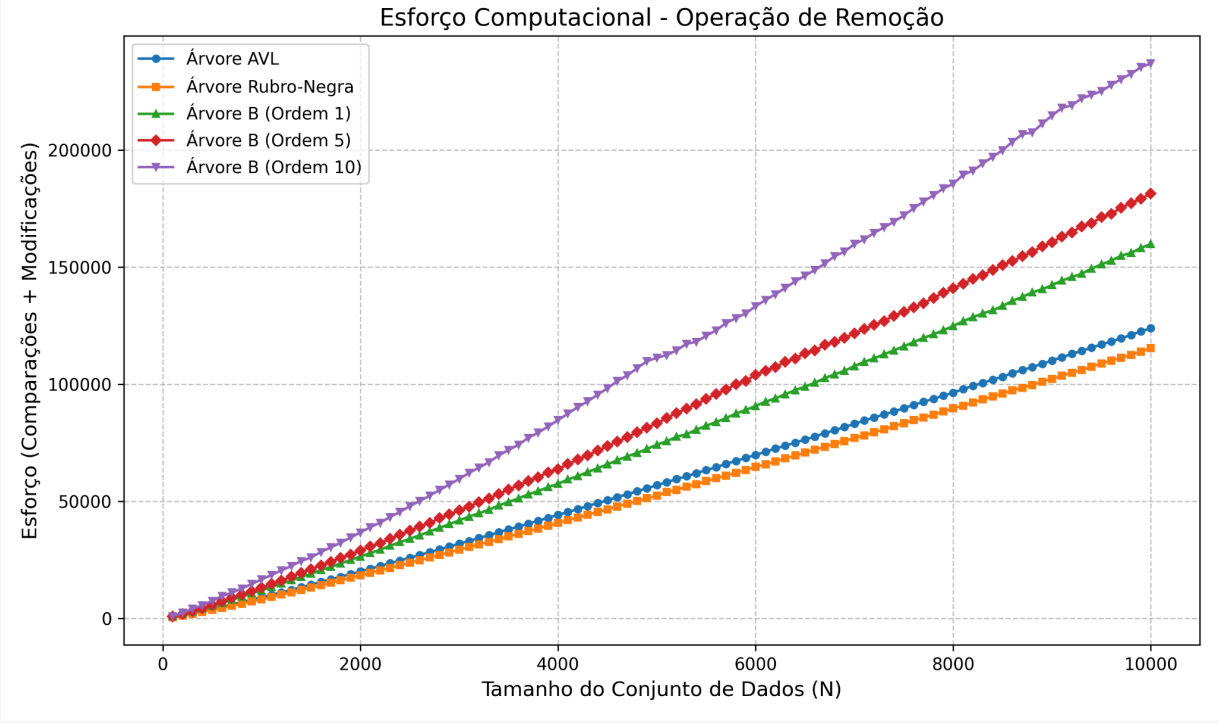
Figura 1 – Esforço computacional médio da operação de inserção para N = 100 a 10.000.

O gráfico da operação de inserção (Figura 1) apresenta o crescimento do esforço total em função de N para as cinco estruturas avaliadas. Observa-se o crescimento aproximadamente linear-logarítmico esperado teoricamente para todas as estruturas, consistente com a complexidade O(n log n) para a inserção sequencial de N elementos.

A Árvore AVL tende a apresentar esforço ligeiramente superior ao da Rubro-Negra nas inserções, pois seu critério de balanceamento mais restrito exige rebalanceamento mais frequente. A Rubro-Negra, por sua vez, amortiza boa parte do custo de balanceamento via recolorações, reduzindo o número efetivo de rotações realizadas.

Entre as Árvores B, a variação de ordem impacta significativamente o comportamento: a Árvore B de ordem 1 (nós com no máximo 2 chaves) realiza splits com maior frequência, pois os nós enchem rapidamente. As ordens 5 e 10 permitem acomodar mais chaves por nó, reduzindo o número de splits e, consequentemente, o esforço de modificação estrutural, ainda que o esforço de comparação dentro de cada nó aumente com a ordem.

3.2 Operação de Remoção



[Gráfico 2: Esforço Computacional – Operação de Remoção]

Figura 2 – Esforço computacional médio da operação de remoção para N = 100 a 10.000.

O gráfico da operação de remoção (Figura 2) revela comportamentos distintos dos observados na inserção. A operação de remoção é geralmente mais custosa do que a inserção em todas as estruturas avaliadas, pois exige localização do elemento, possível substituição por predecessor ou sucessor e propagação do rebalanceamento em direção à raiz.

Na Árvore AVL, a remoção pode provocar rotações em múltiplos nós ao longo do caminho da folha até a raiz, tornando-a comparativamente mais custosa. A Rubro-Negra, mesmo com a complexidade adicional dos casos de nó duplo-preto, realiza no máximo 3 rotações por remoção, tornando seu comportamento mais previsível e controlado.

Para as Árvores B, a remoção pode exigir redistribuição de chaves entre nós irmãos ou fusão de nós, operações que aumentam com a frequência inversamente à ordem da árvore. A Árvore B de ordem 1 realiza fusões com muito mais frequência do que as de ordem 5 ou 10, resultando em maior esforço total de modificação estrutural. Entretanto, as ordens maiores realizam mais comparações por nó durante a fase de localização.

4. Discussão dos Resultados

4.1 Comparação entre AVL e Rubro-Negra

Os resultados experimentais evidenciam as diferenças práticas entre as duas principais árvores binárias balanceadas. A Árvore AVL mantém um balanceamento mais estrito (altura ≤ 1,44 $\log_2 N$), o que se traduz em menor número de comparações em operações de busca. No entanto, esse rigor implica maior número de rotações por operação de modificação.

A Rubro-Negra apresenta balanceamento mais relaxado (altura ≤ 2 $\log_2(N+1)$), mas compensa com um custo de rebalanceamento menor por operação, especialmente nas remoções. Para aplicações onde inserções e remoções são dominantes (como índices de bancos de dados e sistemas de arquivos), a Rubro-Negra tende a ser preferível. Para aplicações de leitura intensiva, a AVL pode oferecer vantagem pela menor altura.

O experimento confirma que, em caso médio com chaves aleatórias, as diferenças entre AVL e Rubro-Negra são relativamente pequenas, mas consistentes com o esperado teoricamente. A Rubro-Negra tende a exibir esforço ligeiramente menor na inserção, enquanto o comportamento na remoção pode variar conforme a distribuição das chaves.

4.2 Impacto da Ordem na Árvore B

A variação da ordem da Árvore B revela uma relação de troca (trade-off) clara entre esforço de comparação e esforço de modificação estrutural. Ordens maiores aumentam a capacidade de cada nó, reduzindo a frequência de splits na inserção e de merges na remoção, mas exigem percorrer mais chaves em cada nó durante a busca.

A Árvore B de ordem 1, com nós de capacidade mínima, apresenta o comportamento mais próximo de uma árvore binária balanceada, com muitos splits e merges. As ordens 5 e 10 demonstram que o aumento do fator de ramificação reduz efetivamente a altura da árvore e o número de modificações

estruturais, mas o esforço de comparação por operação cresce linearmente com a ordem dentro de cada nó.

4.3 Considerações sobre a Validade Estatística

A utilização de 10 amostras independentes por valor de N, cada uma com embaralhamento independente das chaves de inserção e remoção, garante validade estatística adequada para capturar o comportamento de caso médio. O algoritmo de Fisher-Yates assegura que cada permutação das chaves seja igualmente provável, eliminando vieses de ordenação.

O intervalo de N de 100 a 10.000 em incrementos de 100 permite observar com clareza o crescimento logarítmico das curvas. Para valores pequenos de N, o custo adicional para o rebalanceamento pode ser proporcionalmente maior, mas o comportamento diminui à medida que N cresce.

4.4 Escolha da Estrutura por Cenário de Uso

Com base nos resultados e na fundamentação teórica, podem-se fazer as seguintes recomendações práticas:

- Árvore AVL: recomendada para aplicações onde as buscas são muito mais frequentes do que modificações, pois a menor altura garante menos comparações nas pesquisas;
- Árvore Rubro-Negra: recomendada para aplicações com alto volume de inserções e remoções;
- Árvore B (ordem alta): recomendada para sistemas de gerenciamento de bancos de dados e sistemas de arquivos, onde o custo de acesso a disco é dominante e maximizar o fator de ramificação é essencial;
- Árvore B (ordem baixa, t=1 ou t=5): pode ser considerada para memória interna quando se deseja uma estrutura mais genérica que a árvore binária, com melhor comportamento de cache.

5. Conclusão

Este experimento analisou empiricamente a complexidade algorítmica das operações de inserção e remoção em cinco configurações de estruturas de dados balanceadas: Árvore AVL, Rubro-Negra e Árvore B nas ordens 1, 5 e 10. Os resultados confirmam as previsões teóricas de crescimento O(n log n) para a inserção e remoção sequencial de N elementos.

A Árvore AVL demonstrou maior custo de rebalanceamento por operação de modificação em comparação à Rubro-Negra, especialmente nas remoções, onde rotações podem se propagar por múltiplos níveis. A Rubro-Negra, com seu mecanismo de recoloração de custo constante, apresentou comportamento mais uniforme e eficiente nas operações de escrita.

As variações da Árvore B revelaram o impacto direto do parâmetro de ordem no equilíbrio entre esforço de comparação e esforço de modificação estrutural. Ordens maiores reduzem a frequência de splits e merges à custa de maior esforço de comparação intra-nó, evidenciando que a escolha da ordem deve considerar o contexto de uso (especialmente a razão entre custo de acesso a nós e custo de comparação).

É correto afirmar que não existe uma estrutura universalmente superior: a escolha adequada depende do padrão de acesso da aplicação, da proporção entre operações de leitura e escrita, e das características do sistema de armazenamento utilizado. O conhecimento empírico e teórico dessas estruturas é fundamental para o projeto de sistemas computacionais eficientes.